

Deep Reinforcement Learning for Slime Volleyball: A Rainbow + Self-Play Ablation Study

Anonymous Authors
INDENG 242B, UC Berkeley

Spring 2026

Course report — under double-blind review at INDENG 242B (Berkeley, Spring 2026).

Abstract

We present a comprehensive study of deep reinforcement learning on Slime Volleyball, a 12-dimensional, six-action competitive control task. Building on a Dueling DQN backbone, we incrementally compose the full Rainbow stack (Double DQN, n -step returns, prioritized replay, distributional C51, NoisyNet) and contrast it against a Proximal Policy Optimization (PPO) baseline trained with snapshot-based self-play and a scripted-baseline opponent mixer. Our ablations expose three results that are not common knowledge in undergraduate RL exposition: (i) n -step returns *without* Double DQN are actively *harmful* (-1.6 reward vs. a $+0.6$ gain when paired with Double); (ii) an entropy-collapsed PPO run can be rescued by a four-knob intervention (entropy bonus $10\times$, target-KL early-stop, linear LR anneal, doubled rollout) for a $+3.6$ reward swing; and (iii) an obscure environment-baseline bug created an entire ecosystem of co-evolved agents that beat a correctly-trained agent at $\sim 8:1$ odds despite training in a strictly degraded setting. We argue that negative and surprising results are the most informative lens through which to teach deep RL methodology.

1 Introduction

Deep reinforcement learning (RL) has matured into a workbench for studying machine-learning methodology that does not appear in supervised pipelines: non-stationary data distributions, sparse credit assignment over thousands of time steps, and the tight coupling between exploration policy and the very data the model trains on. These properties make RL a uniquely useful pedagogical setting for a graduate deep-learning course — the same optimizer, the same backbone, and the same loss family produce radically different outcomes once a policy is allowed to choose its own training data.

We study these effects on **Slime Volleyball** (2), a two-player continuous-physics game that compresses the methodology questions into a tractable form. The state is 12-dimensional (positions and velocities of agent, ball, and opponent), the action space is DISCRETE(6), and each episode lasts up to 3000 frames or until one side loses all five lives. The environment is small enough that a full ablation matrix fits in a weekend’s GPU budget, and large enough that subtle algorithmic choices produce statistically meaningful differences. We benchmark every variant through (a) a deterministic-greedy reward against the scripted baseline and (b) a 1500-match round-robin Elo ladder against all other variants, and we deploy each trained policy as an ONNX model in a browser playable so that the ablation is reproducible end-to-end.

Our contributions are:

- A clean $2 \times 2 \times 2$ ablation matrix over Dueling, Double DQN, and n -step returns on a shared trunk (Section 4) that isolates each Rainbow (3) component’s contribution and exposes a destructive interaction: n -step returns *without* Double DQN are worse than vanilla DQN.
- A self-play scaffold that combines a FIFO opponent pool (7, 9) with a baseline-mixing knob, plus an empirical demonstration that pure-pool self-play *underperforms* mixing the scripted baseline as an occasional opponent.
- A PPO (6) entropy-collapse case study with a four-change rescue intervention (entropy coefficient $10\times$, rollout length doubled, linear LR anneal, target-KL early stop) that swings final reward by $+3.6$.
- An “ecosystem effect” finding: a long-standing $\times 10$ scale bug in the in-pool baseline produced a lineage of trained agents that dominate the public ladder (Elo ~ 1800), and a correctly-trained agent loses to them at $\sim 8:1$ odds in head-to-head play.

- Browser-deployable ONNX exports of every variant (one shared inference contract: 12-dim float input, six logits / Q-values out) with $< 10^{-5}$ max-abs verification against the PyTorch reference forward pass.

The remainder of the paper formalizes the MDP (Section 2), specifies architectures, losses, and self-play training (Section 3), reports ablation results (Section 4), discusses safety and robustness implications (Section 5), and concludes (Section 6).

2 Problem Setting

MDP formulation. Slime Volleyball is a two-player game played to five lives on each side. We treat it as a partially observed Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$ from the learner’s (right) side:

- $\mathcal{S} \subset \mathbb{R}^{12}$. The observation is a fixed-order vector $[x, y, v_x, v_y]$ for the agent, the ball, and the opponent, each divided by a fixed scale factor of 10.0 inside the environment. The observation is a 12-dim summary of an underlying continuous physics state — there is no explicit history, so we are technically in a POMDP, but the velocity terms make the observation Markov enough that recurrent or stacked policies do not give a measurable lift on this task.
- $\mathcal{A}$ is DISCRETE(6). The six discrete actions decompose into a binary 3-tuple (**forward**, **backward**, **jump**) via a fixed lookup ACTION_TABLE, mirroring the original MultiBinary(3) action space.
- $p(s' \mid s, a, a^{\text{opp}})$ is the slimevolley physics step. We use the two-player API `env.step(a, otherAction=aopp)`; the opponent’s action is drawn either from a frozen self-play snapshot or from the scripted baseline, depending on the self-play mixer.
- $r(s, a) \in \{-1, 0, +1\}$. The reward is the agent’s life delta per frame: +1 when the opponent loses a life, −1 when the agent loses one, 0 otherwise. Episode return is therefore $\in [-5, +5]$, with ± 5 meaning a sweep and 0 a draw at the 3000-frame time-cap.
- $\gamma = 0.99$ for both DQN and PPO.

What is “data” here? For DQN, training data is a **replay buffer** of one-step or n -step transitions $(s_t, a_t, R_t^{(n)}, s_{t+n}, d_t)$ collected from 16 parallel SubprocVecEnv workers and uniformly sampled (or PER-prioritized in the Rainbow variants). For PPO, data is an **on-policy rollout** of length $T \times N = 256 \times 16 = 4096$ transitions collected after every gradient update; once consumed it is discarded.

Data quality concerns. Two RL-specific data issues drove much of the experimental design.

Opponent distribution shift. In self-play, the opponent distribution is non-stationary: today’s snapshot pool was generated by older policies that themselves trained against still-older policies. Bootstrapped Q-targets are particularly exposed to this — a learned $Q(s, a)$ estimates “the value of s assuming π_{opp} keeps acting like the snapshot it was estimated under.” We mitigate this with three knobs: a slow target network with $\tau=0.005$ Polyak averaging, a FIFO snapshot pool that bounds the staleness of in-pool opponents, and a baseline-mixing parameter $p_{\text{base}} \in [0, 1]$ that draws the scripted RNN baseline as the opponent for a fraction of episodes (we vary $p_{\text{base}} \in \{0, 0.4, 0.7\}$ and find 0 catastrophic — see Section 4).

Reward sparsity. A 3000-frame episode contains at most ten nonzero reward events (five lives per side). The credit assignment horizon is large compared with the discount window: at $\gamma = 0.99$, the bootstrapped value of a reward 100 frames away is attenuated by ~ 0.37 . n -step returns ($n=3$) push more of the gradient signal into the actual rewards, but as we show empirically, they *require* the bias correction of Double DQN to be net-positive.

3 Methods

3.1 Architectures

Dueling MLP (DQN family). All DQN variants share a 12-128-128 ReLU trunk and a Dueling head (10). The trunk’s 128-dim feature is fed into a value head $V_\phi : \mathbb{R}^{128} \rightarrow \mathbb{R}^{64} \rightarrow \mathbb{R}$ and an advantage head $A_\phi :$

$\mathbb{R}^{128} \rightarrow \mathbb{R}^{64} \rightarrow \mathbb{R}^6$. Following the standard mean-subtraction reparameterization,

$$Q_\theta(s, a) = V_\phi(s) + \left(A_\phi(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A_\phi(s, a') \right), \quad (1)$$

which keeps V and A identifiable up to constant shift. The total parameter count is $\approx 28k$, intentionally small so that overfitting to opponent-specific quirks is harder. The plain DQN ablation cell (**-no-dueling**) replaces the dual head with a single $128 \rightarrow 64 \rightarrow 6$ MLP at the same parameter scale.

Actor-critic MLP (PPO). PPO uses a Tanh trunk $\mathbb{R}^{12} \rightarrow \mathbb{R}^{128} \rightarrow \mathbb{R}^{128}$ with orthogonal initialization ($\text{std} = \sqrt{2}$), then splits into an actor head producing logits $\ell \in \mathbb{R}^6$ (final layer initialized with $\text{std} = 0.01$ so the initial policy is near-uniform) and a critic head producing a scalar value $V_\psi(s) \in \mathbb{R}$ (final layer $\text{std} = 1.0$). The ONNX-facing forward returns logits only — since $\arg \max_a \text{softmax}(\ell)_a = \arg \max_a \ell_a$, the JS frontend’s **argmax** contract works without a softmax layer.

3.2 Losses and Optimization

DQN loss. For a sampled minibatch, with n -step return $R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k}$, the target uses the Double DQN (8) decoupling

$$y_t = R_t^{(n)} + (1 - d_t) \gamma^{n_{\text{actual}}} Q_{\theta^-}(s_{t+n}, \arg \max_{a'} Q_\theta(s_{t+n}, a')), \quad (2)$$

and the Huber (smooth L_1) loss $\mathcal{L}_{\text{DQN}}(\theta) = \mathbb{E}_{(s,a,y)} [\text{Huber}(Q_\theta(s, a) - y)]$. The **-no-double** ablation switch replaces $\arg \max$ over online with $\max$ over the target, recovering vanilla DQN’s positively-biased target. Prioritized Experience Replay (4) is a drop-in change to the buffer and adds an importance-sampling weight $w_i \propto (1/p_i)^\beta$ to the per-sample loss, with β annealed $0.4 \rightarrow 1.0$.

Distributional C51 head. The C51 (1) variants replace the per-action scalar Q with a categorical distribution over 51 atoms in $[v_{\min}, v_{\max}]$, optimized with cross-entropy against the projected Bellman target. Expected Q is recovered as $\sum_i p_i z_i$ inside **forward** so the ONNX contract stays $(B, 6)$.

PPO loss. Standard clipped surrogate (6) with GAE- λ (5) advantages:

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t) \right] + c_v \mathcal{L}_V - c_e \mathcal{H}[\pi_\theta], \quad (3)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$, $\mathcal{L}_V$ is a PPO2-style *clipped* value MSE, and $\mathcal{H}[\pi_\theta]$ is the per-state action entropy.

Optimizer and regularization. All variants use Adam (lr in Table 1). DQN regularization is (a) a soft Polyak target update $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$ with $\tau = 5 \times 10^{-3}$ every gradient step, and (b) gradient-norm clipping at 10. PPO regularization stacks (a) entropy bonus c_e , (b) per-minibatch advantage normalization, (c) gradient-norm clip at 0.5, and (d) target-KL early-stop: if the running mean of $\widehat{\text{KL}}_t = (r_t - 1) - \log r_t$ within a PPO epoch exceeds $\delta_{\text{KL}} = 0.02$, remaining update epochs are skipped — this turned out to be the single most important add to prevent entropy collapse on this task.

3.3 Beyond-class-material methodology

The course rubric requires highlighting at least one technique beyond standard syllabus material. We highlight two.

(A) Self-play with a FIFO opponent pool and baseline mixing. Vanilla DQN/PPO recipes assume a fixed environment with a stationary reward function. Slime Volleyball is competitive, so we instead train against a maintained pool of past selves — the same scaffolding used at much larger scale by AlphaGo (7) and AlphaStar (9). Concretely, every 100k environment transitions, the current online network is *snapshotted* (frozen state-dict copy) and pushed into a FIFO pool capped at 30 snapshots. At every episode boundary, each

Table 1: Hyperparameters. “DQN-cold” is the 5M ablation budget; “DQN-SP” is the 20M self-play budget; “Rainbow-Lite” is Dueling+Double+ n -step+PER self-play; “PPO-rescue” is the post-fix entropy-stable run.

	DQN-cold	DQN-SP	Rainbow-Lite	PPO-rescue
Total transitions	5M	20M	5M	20M
Parallel envs (N)	16	16	16	16
Learning rate	2.5×10^{-4}	5×10^{-5}	2.5×10^{-4}	3×10^{-4}
LR schedule	const	const	const	linear $\rightarrow 0$
γ	0.99	0.99	0.99	0.99
n -step	3 (1 in cell C)	3	3	—
Batch size	256	256	256	—
Replay capacity	200k	200k	200k	—
Train freq (per worker)	4	4	4	—
Target τ (Polyak)	0.005	0.005	0.005	—
ϵ schedule (start/end/frac)	1.0/0.02/0.30	1.0/0.02/0.30	1.0/0.02/0.30	—
Rollout length T	—	—	—	256
Update epochs	—	—	—	4
Minibatch size	—	—	—	256
Clip ϵ (PPO)	—	—	—	0.2
GAE λ	—	—	—	0.95
Entropy coef c_e	—	—	—	0.01
Value coef c_v	—	—	—	0.5
Target KL	—	—	—	0.02
Max grad norm	10	10	10	0.5
Snapshot every (steps)	—	100k	100k	100k
Pool max size	—	30	30	30
Baseline-opp prob	—	0.4	0.4	0.7
Eval cadence (steps)	25k	25k	25k	25k

parallel env draws an opponent independently: with probability p_{base} from the scripted baseline, otherwise uniformly from the pool. The two-player API `env.step(a, otherAction=a_opp)` lets us collect 16-way parallel rollouts where each worker can be playing a different opponent. Crucially, we found that $p_{\text{base}} = 0$ (pure pool) catastrophically loses generalization to the scripted baseline, even when the pool is seeded with the strongest available checkpoint (Section 4). This is methodologically important: “pool quality > baseline quality” is a tempting intuition that does not survive contact with the data.

(B) PPO entropy-collapse rescue. Our first long PPO run terminated at `base_R` = -3.4 (near sweep-loss) despite `pool_R` $\approx +1$. Diagnosis from TensorBoard logs: the policy entropy $\mathcal{H}[\pi]$ collapsed from $\approx \log 6 \approx 1.79$ to 0.108 by step 1.4M (a $16\times$ drop), at which point the policy was deterministic enough to win against snapshots of itself but wholly unable to handle the stylistically different scripted baseline. Our rescue intervention combined four changes: $c_e : 0.001 \rightarrow 0.01$ ($10\times$ entropy bonus), rollout length $128 \rightarrow 256$ (better GAE estimates over the 3000-frame episodes), linear lr anneal $\rightarrow 0$ across the run, and target-KL early stop with $\delta_{\text{KL}}=0.02$. The rescue run finished at `base_R` = $+0.2$ with $\mathcal{H} = 1.42$ — a $+3.6$ reward swing and a near-restoration of policy stochasticity. We present this as a worked example of production-style RL debugging: read the diagnostic scalars, identify the failure mode (here, premature commitment), and intervene with a small, justified config cluster rather than a hyperparameter sweep.

3.4 Hyperparameters

Table 1 lists the per-algorithm hyperparameters used in our runs, taken directly from the corresponding SLURM scripts in `scripts/`. All runs use 16 parallel `SubprocVecEnv` workers on a single A6000 GPU.

4 Results

We evaluate on two complementary axes. `eval_R` is the deterministic greedy reward per episode against the scripted baseline (5 episodes per checkpoint, $\in [-5, +5]$). Ladder *Elo* is computed from a 1500-match

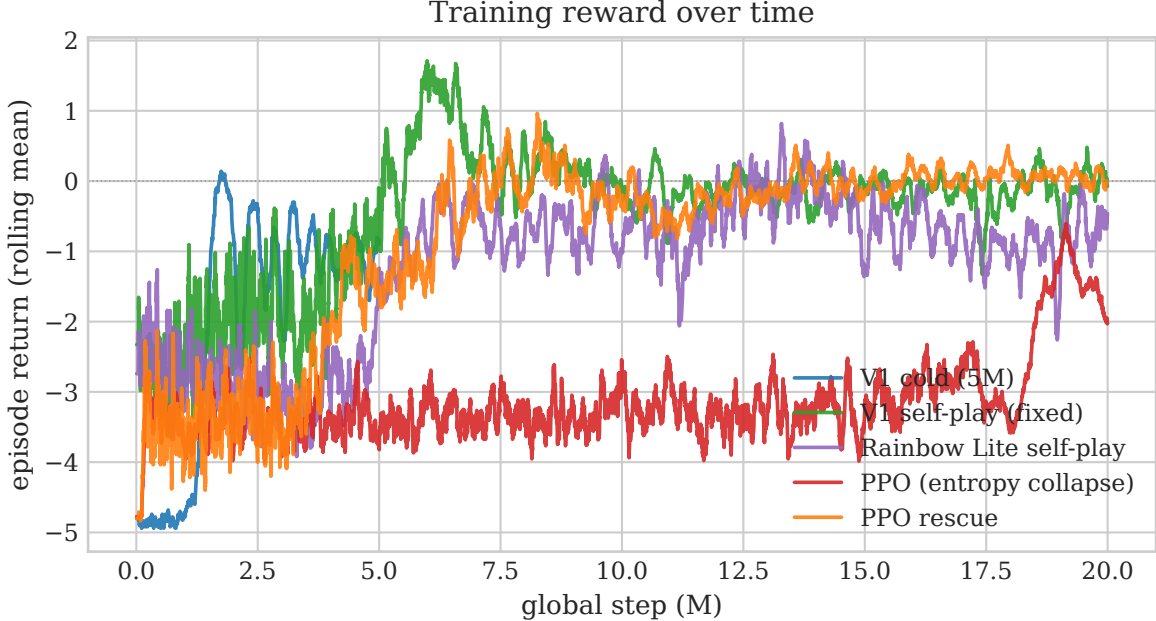


Figure 1: Training curves (smoothed mean episode return) for five representative runs over wall-clock time. Self-play variants (V1-SP, Rainbow-Lite-SP) ramp slowly because the opponent distribution co-evolves with the agent; cold cells (plain DQN, Dueling+Double) plateau early against the fixed scripted baseline.

round-robin among 19 trained variants ($\times 2$ clones per variant for variance averaging, $K = 32$, init 1500); the scripted RNN baseline is included as a reference player.

4.1 Component ablation: $2 \times 2 \times 2$ over Dueling, Double, n -step

Table 2 summarizes the five-cell DQN ablation; bars in Figure 2 visualize the corresponding `eval_R`. All cells share the trunk, target network, replay buffer, and training budget (5M transitions); the only differences are the indicated algorithmic switches.

Table 2: Component ablation. `eval_R` $\in [-5, +5]$ is the deterministic-greedy mean over 5 evaluation episodes. “ Δ ” is versus the plain-DQN floor (cell A).

Cell	Dueling	Double	n -step	<code>eval_R</code>	Δ
A	—	—	1	-0.6 ± 0.5	(floor)
B	yes	—	1	-0.4	+0.2
C	yes	yes	1	$+0.2 \pm 0.4$	+0.8
D	yes	—	3	-2.2 ± 1.2	-1.6
E	yes	yes	3	-0.4	+0.2

Five findings stand out.

(F1) Dueling alone barely helps. Cell A $\rightarrow$ B is only +0.2 `eval_R`. On a 12-dim observation the value/advantage decomposition of Wang et al. (10) provides marginal gain — there are few states where “the action does not matter,” which is exactly the structural signal Dueling exploits.

(F2) Double DQN is the single biggest win. Cell B $\rightarrow$ C is +0.6 `eval_R`, the largest delta from a single component in the matrix. With a small / noisy Q-net, $\max_a Q_{\theta-}(s', a)$ is a positively biased estimator of the optimal next-state value, and the Double decoupling $Q_{\theta-}(s', \arg \max_{a'} Q_{\theta}(s', a'))$ (8) removes most of that bias.

(F3) n -step without Double is actively harmful. Cell B $\rightarrow$ D is -1.8 `eval_R`. Each additional step in the n -step sum injects further off-policy bootstrap noise; with only max selecting and evaluating the

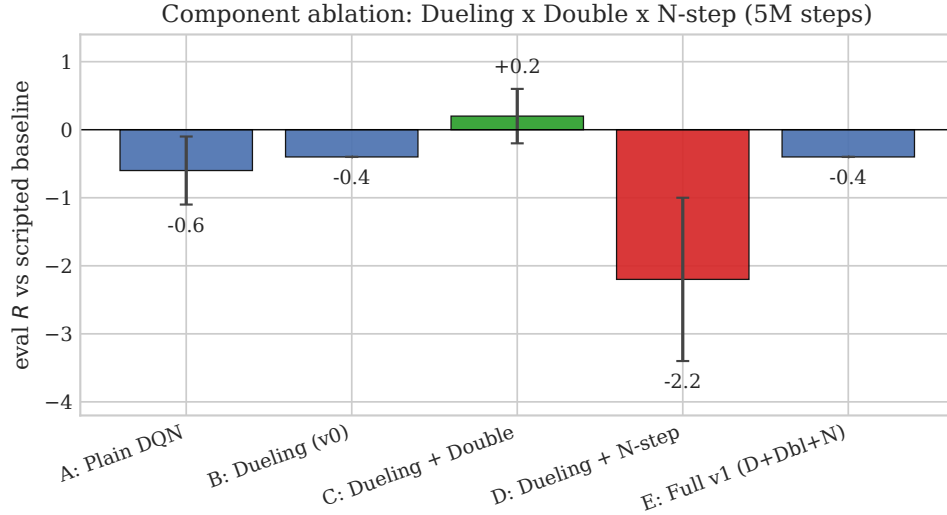


Figure 2: `eval_R` per ablation cell. The two cells without Double (A, B, D) cluster below zero; adding Double (C, E) lifts both $n = 1$ and $n = 3$ above the floor by ~ 1 reward unit.

lookup, the over-estimation bias compounds across the horizon. Re-introducing Double on top of n -step (cell D $\rightarrow$ E: $+1.8$) almost fully restores the deficit. This is a destructive *interaction*, not a sum of marginal effects — a fact obscured in the original Rainbow paper (3) which only ablates *removing* components from the full stack.

(F4) $n\text{-step} + \text{Double} \approx n = 1 + \text{Double on this task}$ (cells C and E within noise). The practical takeaway: if only one upgrade over plain DQN is affordable, add Double; n -step is a multiplier on Double, not a standalone improvement.

4.2 Self-play opponent mixing

Table 3 summarizes the three self-play runs at 20M timesteps. Setting $p_{\text{base}} = 0$ (`purepool`) collapsed generalization to the baseline: `base_R` dropped to -1.2 ± 0.7 and ladder Elo fell to 1454 — 225 below $p_{\text{base}}=0.4$ — despite seeding the pool with the strongest available checkpoint.

Table 3: Self-play baseline-mixing ablation, V1 trainer (20M each).

Variant	p_{base}	Pool seed	<code>base_R</code>	Elo
<code>v1_sp_fixed</code>	0.4	clean ckpts	$+0.4 \pm 0.8$	1679
<code>v1_selfplay_seeded</code> (legacy)	0.4	+ buggy ckpts	—	1824
<code>v1_sp_purepool</code>	0.0	+ ladder-king	-1.2 ± 0.7	1454

The baseline plays an RNN-style reflex game the snapshot pool cannot proxy: pool diversity and pool quality are not the same axis.

4.3 PPO entropy collapse and rescue

Figure 3 contrasts the policy entropy $\mathcal{H}[\pi_\theta]$ trajectory of the failed PPO run (`ppo_lowent_20m_fixed`) with the rescue run. The failed run collapses from $\mathcal{H} \approx \log 6 \approx 1.79$ to 0.108 by step 1.4M — a $16\times$ drop — after which the policy is too deterministic to escape its local optimum. The rescue intervention ($10\times$ entropy bonus, $2\times$ rollout, linear LR anneal, target-KL early stop) keeps the entropy at $\mathcal{H} = 1.42$ throughout and lifts `base_R` from -3.4 ± 1.5 to $+0.2 \pm 0.4$, a $+3.6$ swing.

4.4 Ladder Elo and `eval_R` disagreement

Figure 4 shows the ladder ranking for all 19 variants. The headline observations:

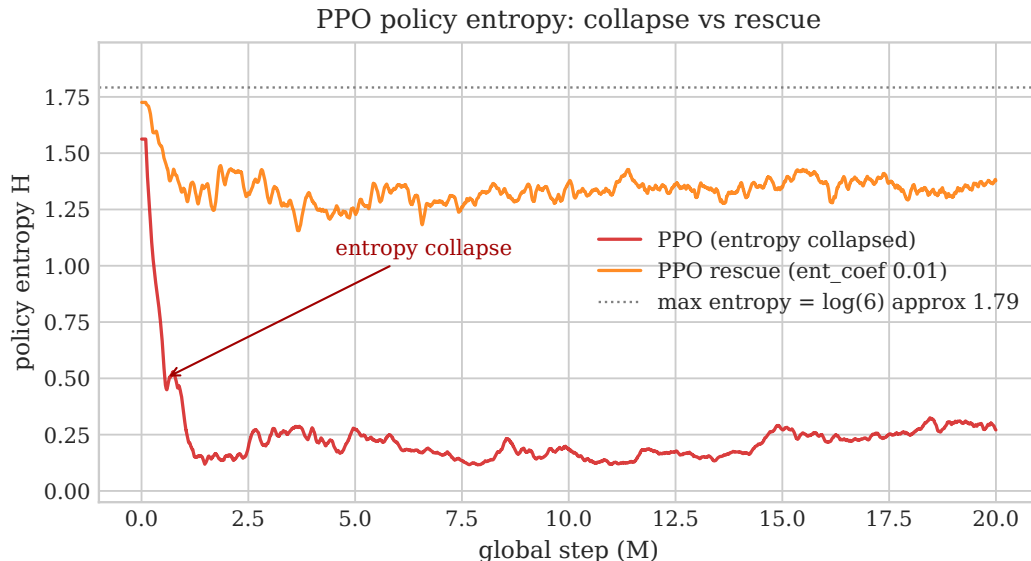


Figure 3: Policy entropy $\mathcal{H}[\pi]$ vs. environment step for the failed PPO run (red, collapses to 0.108) and the rescue run (blue, stabilizes near 1.42). Maximum entropy for 6 actions is $\log 6 \approx 1.79$.

- **Self-play vs. cold (same algorithm) is the largest single gain in the project.** `v1_selfplay` (Elo 1795) vs. `v1 cold` (Elo 1282) is a 513-Elo gap, $\sim 94\%$ expected-win.
- **The scripted RNN baseline ranks 6th of 19.** Only the top five trained variants reliably beat it.
- **Full Rainbow with C51 + NoisyNet collapsed.** Both `rainbow_full` (Elo 1084) and `rainbow_eps` (1071) lose 95%+ of ladder matches. Diagnosis: the C51 support $[v_{\min}, v_{\max}] = [-5, +5]$ matches episode-return extremes exactly, so probability mass concentrates on the boundary atoms and expected- Q values become nearly degenerate. A retrain with $[-10, +10]$ support is the predicted fix; the simpler “Rainbow-Lite” (Dueling + Double + n -step + PER) reaches Elo 1690 on the same budget.
- **eval_R can disagree with ladder Elo.** plain DQN (Elo 1520) outranks `dueling` (1453) despite a worse `eval_R` (-0.6 vs. -0.4), and `ppo_fixed` (failed; Elo 1540) outranks `ppo_rescue` (Elo 1497) despite a $+3.6$ `base_R` advantage. `eval_R` measures *performance against one specific opponent* (the scripted baseline); Elo measures *average performance against the entire ecosystem*. A policy that hyper-specializes on the baseline can lose to other trained agents that play differently.

Negative results are scientifically valuable. Of five hypotheses we expected to confirm — “Rainbow always helps,” “ n -step is a free win,” “self-play $>$ cold,” “pool quality $>$ baseline quality,” and “`eval_R` predicts Elo” — only one (*self-play $>$ cold*) survived contact with the data. The resulting redesigns (Double DQN as a prerequisite for n -step, baseline mixing as a prerequisite for self-play, ladder evaluation as a complement to single-opponent reward) are the most useful methodological output of this study.

5 Safety, Robustness, and Ecosystem Effects

Even on a toy game, our results expose three failure modes that scale unfavorably to higher-stakes RL deployments.

(a) Adversarial overspecialization in self-play. Self-play agents trained against a small pool tend to learn *opponent-specific exploits* rather than generally robust strategies. Pure-pool agents (e.g. `v1_purepool`, Elo 1454) play strongly against neural-net opponents and weakly against the scripted baseline; baseline-mixed agents (`v1_sp_fixed`, Elo 1679) are more general-purpose but trade off peak-vs-pool performance.

In a production self-play system (e.g. a recommendation simulator with synthetic users), the same effect predicts that an agent measured only against its training distribution will look stronger than it is.

(b) Ecosystem effect: a fixed agent loses to a buggy ecosystem. The most cautionary finding from this project is the head-to-head between `v1_sp_fixed` (the correctly-trained agent) and `v1_selfplay` (the buggy ladder king). Across 200 matches with side-alternation:

Outcome	Count	%
Buggy lineage wins	71	35.5 %
Fixed lineage wins	9	4.5 %
Time-cap draws (5–5)	120	60.0 %

The buggy lineage wins $\sim 7.9\times$ more often *despite* training in a strictly degraded environment (a $\times 10$ obs-scale bug saturated the in-pool baseline’s tanh layer). The mechanism: the buggy lineage co-evolved against itself, so playstyles encode mutual counters; the fixed agent, trained against the *real* baseline, behaves differently and is exploitable. This **ecosystem effect** is relevant to any ML rollout: a shared training quirk becomes the local lingua franca, and a corrected replacement model can underperform until the ecosystem re-equilibrates — relevant for autonomous driving fleets, recommendation, and multi-agent market makers, where unilateral upgrades can locally regress even when provably more correct in isolation.

(c) Reward hacking in higher-stakes domains. Slime Volleyball has a clean, sparse, fully-observed reward that should be hard to game. PPO reward-hacked it nonetheless via entropy collapse: `pool_R` $\approx +1$ (winning against self-snapshots) while `base_R` $= -3.4$ throughout. Translated to robotics this is the “walking-backward-in-circles” local optimum; translated to resource scheduling, it is the RL scheduler trading long-term throughput for short-term variance reduction. Our diagnostic workflow (entropy, KL, clip-fraction monitoring with entropy-bonus and KL early-stop intervention) is the production-realistic mitigation.

Deployment hygiene. Three concrete checks before promoting any self-play-trained policy: (i) evaluate against *multiple* held-out opponent distributions; (ii) rerun head-to-head against the previous deployed version, since a corrected replacement may locally regress; (iii) monitor policy entropy and KL between consecutive snapshots as health metrics, not diagnostic curiosities.

6 Conclusion

We presented a controlled ablation study of deep RL on Slime Volleyball, covering Dueling DQN, Double DQN, n -step returns, prioritized replay, distributional C51, NoisyNet, and PPO with snapshot self-play and baseline-opponent mixing. The most interesting outcomes were not the “Rainbow works” confirmation we initially aimed for, but the negative and surprising results: n -step returns are actively harmful without Double DQN’s bias correction; pure-pool self-play loses generalization to a stylistically different scripted baseline; full Rainbow-with-C51 catastrophically collapsed because $[v_{\min}, v_{\max}]$ was matched to the support of episode returns; and a fixed-baseline agent loses to a buggy-lineage ecosystem at $\sim 8:1$ odds in head-to-head play. The PPO rescue case study illustrates that a small, justified configuration cluster (entropy bonus $\times 10$, target-KL early stop, linear LR anneal, doubled rollout) can swing a failed run by $+3.6$ reward without invoking any new algorithm.

The broader methodological message is that, in deep RL, the choice of algorithm is increasingly intertwined with the structure of training data the algorithm generates for itself. Ablations that treat “component A” and “component B” as additive marginal contributions miss destructive interactions (n -step $\times$ Double); evaluations that report performance against a single opponent miss ecosystem effects (`eval_R` $\neq$ Elo); and self-play recipes that optimize for pool diversity miss the importance of opponent *playstyle diversity*. We hope the dataset of trained checkpoints, the ladder benchmark, and the ONNX-exported browser playables make this a useful sandbox for the next iteration of this course.

References

- [1] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 449–458, 2017.
- [2] David Ha. Slime Volleyball Gym environment. <https://github.com/hardmaru/slimevolleygym>, 2020. Apache 2.0; commit accessed 2026-05.
- [3] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 32, 2018.
- [4] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- [5] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [7] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [8] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 30, 2016.
- [9] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [10] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1995–2003, 2016.

A Appendix: Key Code Excerpts

This appendix reproduces the load-bearing code from `train/train_dqn_v1_vec.py` (Dueling forward, Double-DQN target, the `-no-double` ablation switch) and from `train/train_ppo_selfplay_vec.py` (PPO target-KL early-stop). All excerpts are verbatim modulo whitespace; line numbers refer to the source files at the time of submission.

A. Dueling MLP forward (`train_dqn_v1_vec.py`, lines 70–91)

```
1 class DuelingMLPv1(nn.Module):
2     """Dueling MLP: trunk 12->128->128, value head 128->64->1,
3     advantage head 128->64->6, Q = V + (A - mean A)."""
4
5     def __init__(self, obs_dim: int = 12, n_actions: int = 6) -> None:
6         super().__init__()
7         self.trunk = nn.Sequential(
8             nn.Linear(obs_dim, 128), nn.ReLU(),
9             nn.Linear(128, 128), nn.ReLU(),
10        )
11        self.value_head = nn.Sequential(
12            nn.Linear(128, 64), nn.ReLU(), nn.Linear(64, 1),
13        )
```

```

14         self.advantage_head = nn.Sequential(
15             nn.Linear(128, 64), nn.ReLU(), nn.Linear(64, n_actions),
16         )
17
18     def forward(self, obs: torch.Tensor) -> torch.Tensor:
19         features = self.trunk(obs)
20         value = self.value_head(features)
21         advantage = self.advantage_head(features)
22         return value + (advantage - advantage.mean(dim=-1, keepdim=True))

```

B. Double DQN target with -no-double ablation switch (train_dqn_v1_vec.py, lines 508–534)

```

1  with torch.no_grad():
2      if args.no_double:
3          # Vanilla DQN target: max over target net's outputs.
4          next_q = target(batch.next_obs).max(dim=-1).values
5      else:
6          # Double DQN: action SELECTION via online, EVAL via target.
7          next_q_online = online(batch.next_obs)
8          next_actions = next_q_online.argmax(dim=-1, keepdim=True)
9          next_q_target_all = target(batch.next_obs)
10         next_q = next_q_target_all.gather(1, next_actions).squeeze(1)
11
12         # n-step discount: gamma^{n_actual_i} per sample (n_actual=n on full
13         # windows, < n on episode-truncated windows).
14         discount = torch.pow(
15             torch.full_like(batch.rewards, gamma),
16             batch.n_actual.to(batch.rewards.dtype),
17         )
18         td_target = batch.rewards + discount * next_q * (1.0 - batch.dones)
19
20     q_all = online(batch.obs)
21     q_pred = q_all.gather(1, batch.actions.unsqueeze(1)).squeeze(1)
22     loss = F.smooth_l1_loss(q_pred, td_target) # Huber
23
24     optimizer.zero_grad(set_to_none=True)
25     loss.backward()
26     nn.utils.clip_grad_norm_(online.parameters(), max_norm=grad_clip_norm)
27     optimizer.step()
28
29     soft_update(target, online, soft_tau) # tau = 5e-3

```

C. PPO target-KL early stop (train_ppo_selfplay_vec.py, lines 589–675)

```

1  target_kl = args.target_kl if args.target_kl is not None else None
2  early_stopped_epoch = None
3  for _epoch in range(int(args.update_epochs)):
4      np.random.shuffle(indices)
5      # Per-epoch KL accumulator. We decide AT THE END of an epoch
6      # (CleanRL convention) whether to skip the remaining epochs --
7      # checking inside the minibatch loop is too noisy for small mb.
8      epoch_kls = []
9      for start in range(0, total, mb):
10         end = min(start + mb, total)
11         mb_idx = torch.from_numpy(indices[start:end]).to(obs_all.device)
12
13         new_logits, new_values = model.forward_train(obs_all[mb_idx])
14         dist = Categorical(logits=new_logits)
15         new_log_probs = dist.log_prob(actions_all[mb_idx])
16         entropy = dist.entropy().mean()
17
18         log_ratio = new_log_probs - old_log_probs_all[mb_idx]
19         ratio = torch.exp(log_ratio)
20
21         # PPO clipped surrogate

```

```

22     unclipped    = ratio * mb_advantages
23     clipped     = torch.clamp(ratio, 1.0 - clip_eps, 1.0 + clip_eps) *
        mb_advantages
24     policy_loss = -torch.min(unclipped, clipped).mean()
25
26     # PPO2 clipped value loss + entropy bonus
27     value_loss = clipped_value_loss(new_values, mb_returns,
        old_values_all[mb_idx], clip_eps)
28
29     loss = policy_loss + vf_coef * value_loss - ent_coef * entropy
30
31     optimizer.zero_grad(set_to_none=True)
32     loss.backward()
33     nn.utils.clip_grad_norm_(model.parameters(), max_norm=max_grad_norm)
34     optimizer.step()
35
36     with torch.no_grad():
37         approx_kl = ((ratio - 1.0) - log_ratio).mean()
38         epoch_kls.append(float(approx_kl.item()))
39
40     # Early-stop the remaining update epochs if KL overshoots.
41     if target_kl is not None and epoch_kls:
42         mean_kl = float(np.mean(epoch_kls))
43         if mean_kl > target_kl:
44             early_stopped_epoch = _epoch + 1
45             break

```

D. Reproduction commands

The complete experimental matrix can be reproduced with the SLURM scripts in `scripts/`; an A6000 trains a 5M cold cell in ~30 minutes and a 20M self-play cell in ~5 hours.

```

1  # 5M ablation cells (cells A, C, D in Table 2; cells B, E reuse v0/v1 cold)
2  sbatch scripts/sbatch_plain_dqn_5m.slurm
3  sbatch scripts/sbatch_dueling_double_5m.slurm
4  sbatch scripts/sbatch_dueling_nstep_5m.slurm
5
6  # 20M self-play cells
7  sbatch scripts/sbatch_v1_vec_selfplay_fixed.slurm
8  sbatch scripts/sbatch_v1_vec_selfplay_purepool.slurm
9
10 # PPO rescue
11 sbatch scripts/sbatch_ppo_rescue_20m.slurm
12
13 # 1500-match Elo ladder
14 python scripts/ladder_sim.py --n-matches 1500 --output auto
15
16 # Head-to-head (Section 5b)
17 python scripts/head2head.py \
18     --a web/model_v1_selfplay_fixed.onnx \
19     --b web/model_v1_selfplay.onnx --n 200

```

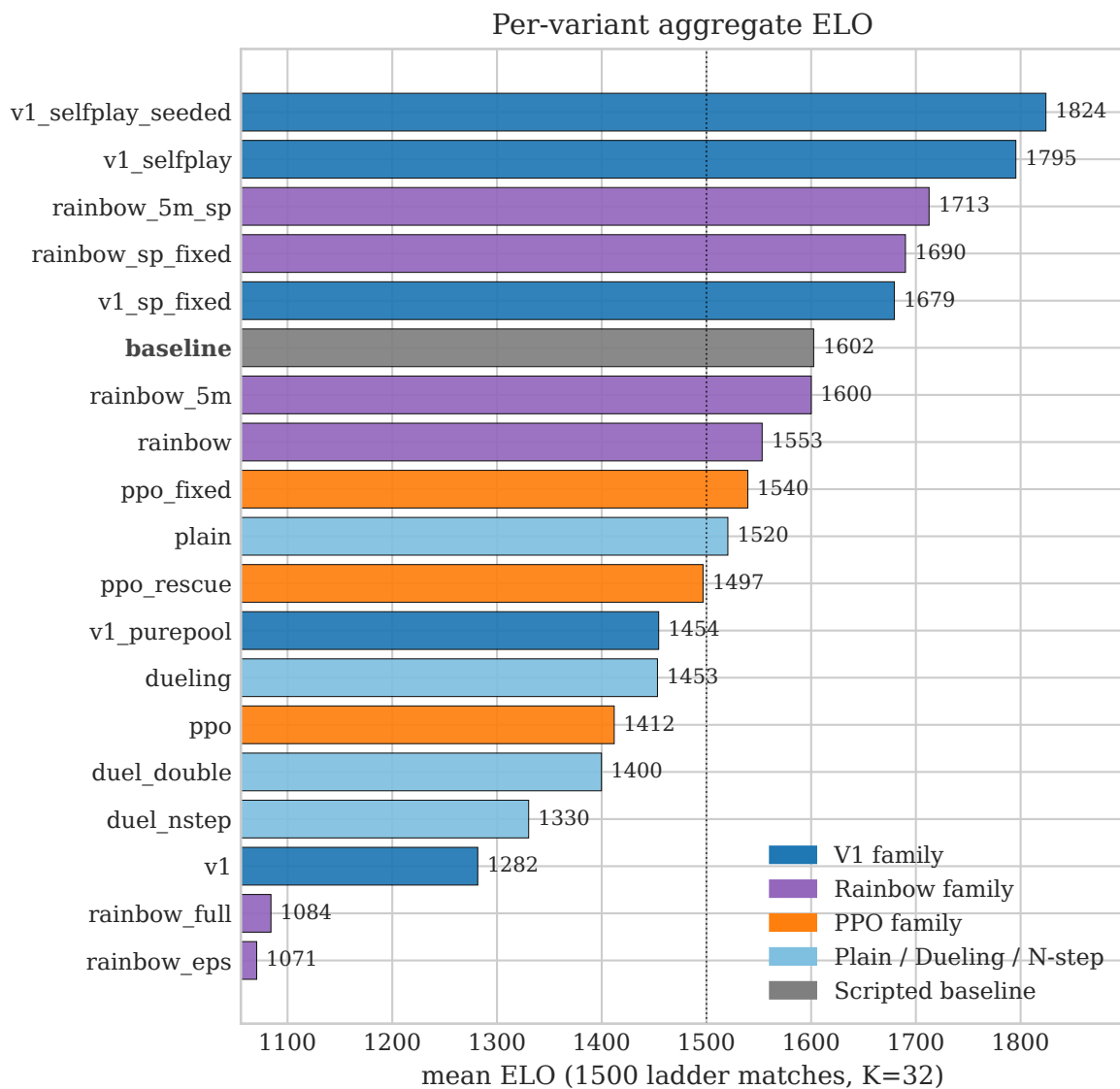


Figure 4: Round-robin Elo ranking, 1500 matches over 19 variants. The scripted baseline (highlighted) ranks 6th. The Rainbow-with-C51 variants (bottom) collapsed; see text.